

SensorLog: Sistema de Gerenciamento e Análise de Sensores

EEL670 – Linguagens de Programação - 26.1

Trabalho – Etapa 2

Aluno: Thiago Marques de Oliveira

DRE: 122053397

Data: Julho, 2026

1 Introdução

O desenvolvimento de sistemas modernos frequentemente exige a combinação de linguagens de programação distintas para aproveitar o melhor de cada ecossistema. Neste contexto, foi desenvolvido o **SensorLog**, um sistema para gerenciamento de sensores IoT (temperatura, umidade e luminosidade). O objetivo é permitir o cadastro de sensores, importação de leituras em massa via CSV, e a realização de análises complexas e geração de gráficos.

O gerenciador principal foi escrito em C++ utilizando o padrão C++17 e o paradigma de orientação a objetos. Por outro lado, para facilitar a análise matemática e a visualização de dados, funções externas foram implementadas em Python, utilizando bibliotecas como `numpy`, `statistics` e `matplotlib`. A comunicação entre as duas linguagens ocorre via `popen()/pclose()` (pipes POSIX), com parâmetros serializados em JSON na linha de comando e retorno via `stdout`, sem o uso de arquivos temporários para troca de parâmetros e sem o uso da função proibida `system()`.

O programa oferece 8 opções ao usuário via menu interativo, das quais 3 utilizam integração com o módulo Python: cálculo de estatísticas descritivas, detecção de anomalias por z-score e geração de gráficos de histórico. Todo o sistema é compilável em Linux através de um `Makefile`.

2 Implementação do Programa

2.1 Critérios de Escolha e Arquitetura

O **Python** foi escolhido para as funções externas pois possui bibliotecas nativas e poderosas para matemática (`statistics`, `numpy`) e plotagem (`matplotlib`), que seriam trabalhosas de implementar em C++ puro.

O projeto é dividido em diretórios lógicos:

- `src/` – Código-fonte C++ (classes e `main.cpp`).
- `include/` – Arquivos de cabeçalho (`.h`).
- `scripts/` – Módulo Python (`analise.py`).
- `data/` – Arquivos de teste (CSV).

O sistema é compilado através de um Makefile utilizando as flags `-Wall -Wextra -pedantic` e o padrão `-std=c++17` para garantir a qualidade do código.

2.2 Classes e Estruturas

O paradigma de orientação a objetos foi utilizado extensivamente. As classes implementadas são:

- **Sensor** – Encapsula o ID e o tipo (`temp/hum/lux`) de um sensor cadastrado.
- **Leitura** – Encapsula o ID da leitura, o ID do sensor, tipo, valor, unidade e timestamp de cada medição.
- **GerenciadorSensores** – Núcleo do sistema. Mantém um `std::map<string, Sensor>` para sensores cadastrados e um `std::vector<Leitura>` para o histórico de leituras. Realiza parsing de CSV, valida entradas e delega tarefas analíticas ao módulo externo.
- **InterfacePython** – Encapsula a comunicação com o script Python via `popen()`. Serializa parâmetros em JSON, lê o JSON retornado via `stdout` e trata erros.

2.3 Interface de Comunicação C++ ↔ Python

A interface entre as duas linguagens foi o ponto crítico do projeto. Para atender à restrição de não usar arquivos para passagem de parâmetros e não usar a função proibida `system()`, optou-se pela função POSIX `popen()`. Ela abre um pipe que permite executar o script Python e capturar seu `stdout`. Os parâmetros são serializados em uma string JSON e passados como argumento de linha de comando para o Python. O resultado é retornado via `stdout`, lido pelo C++ e desserializado.

O trecho abaixo demonstra a implementação da função `executar` da classe `InterfacePython`:

```
1 std::string InterfacePython::executar(const std::string& funcao,
2                                     const std::string& payload) const
3 {
4     std::string payloadEscapado = escaparAspas(payload);
5     std::string cmd = "python3 " + m_scriptPath +
6                     " " + funcao +
7                     " '" + payloadEscapado + "' 2>&1";
8
9     FILE* pipe = popen(cmd.c_str(), "r");
10    if (!pipe)
11        throw std::runtime_error("Falha ao abrir pipe com o modulo
12    Python.");
13
14    std::string resultado;
15    char buffer[512];
16    while (fgets(buffer, sizeof(buffer), pipe) != nullptr)
17        resultado += buffer;
18
19    pclose(pipe);
20    if (resultado.find("\nerro\n") != std::string::npos) {
21        std::string msg = extrairString(resultado, "erro");
22        throw std::runtime_error("Erro no modulo Python: " + msg);
23    }
24    return resultado;
```

24 }

Listing 1: Execução do script Python via popen (InterfacePython.cpp)

2.4 Robustez

Para garantir que o programa não quebre com entradas inválidas, diversas medidas foram tomadas:

- O menu principal rejeita entradas não numéricas, limpando o buffer do `cin`.
- A importação de CSV rejeita linhas mal formatadas individualmente, utilizando blocos `try-catch`.
- Apenas leituras de sensores previamente cadastrados são aceitas.
- Tipos de sensor inválidos são rejeitados com mensagem clara.
- IDs duplicados são detectados e reportados.

```

1 try {
2     valor = std::stod(valorStr);
3 }
4 catch (...) {
5     std::cout << "[AVISO] Valor invalido. Linha ignorada.\n";
6     ++rejeitadas;
7     continue;
8 }

```

Listing 2: Validação de CSV (GerenciadorSensores.cpp)

2.5 Análise Estatística no Python

No lado do Python, o script `analise.py` recebe o nome da função e o payload JSON via `sys.argv`, processa a lógica com `numpy/statistics` e retorna o resultado como JSON no `stdout`.

```

1 def detectar_anomalias(payload: dict) -> dict:
2     valores = payload.get("valores", [])
3     k = float(payload.get("k", 2.0))
4
5     arr = np.array(valores, dtype=float)
6     media = float(np.mean(arr))
7     desvio = float(np.std(arr, ddof=1))
8
9     indices_anomalos = []
10    zscores_anomalos = []
11    for i, v in enumerate(valores):
12        z = (v - media) / desvio
13        if abs(z) > k:
14            indices_anomalos.append(i)
15            zscores_anomalos.append(round(z, 4))
16
17    return {"anomalias": indices_anomalos, "zscore": zscores_anomalos}

```

Listing 3: Detecção de anomalias em Python (analise.py)

2.6 Compilação via Makefile

O projeto utiliza um **Makefile** para automatizar a compilação e o gerenciamento de dependências. Três comandos principais estão disponíveis:

- **make deps** – Instala as dependências Python necessárias (**matplotlib** e **numpy**) via **pip3**.
- **make** – Compila todos os arquivos-fonte C++ do diretório **src/** com as flags **-std=c++17 -Wall -Wextra -pedantic**, gerando o binário **sensorlog**.
- **make clean** – Remove os arquivos objeto do diretório **build/** e o binário gerado.

O conteúdo do **Makefile** é apresentado abaixo:

```
1 CXX      := g++
2 CXXFLAGS := -std=c++17 -Wall -Wextra -pedantic -Iinclude
3 TARGET   := sensorlog
4
5 SRC_DIR  := src
6 OBJ_DIR  := build
7
8 SRCS := $(wildcard $(SRC_DIR)/*.cpp)
9 OBJS := $(patsubst $(SRC_DIR)/%.cpp, $(OBJ_DIR)/%.o, $(SRCS))
10
11 all: $(TARGET)
12
13 $(TARGET): $(OBJS)
14     $(CXX) $(CXXFLAGS) -o $@ $^
15
16 $(OBJ_DIR)/%.o: $(SRC_DIR)/%.cpp | $(OBJ_DIR)
17     $(CXX) $(CXXFLAGS) -c $< -o $@
18
19 $(OBJ_DIR):
20     mkdir -p $(OBJ_DIR)
21
22 deps:
23     pip3 install matplotlib numpy
24
25 clean:
26     rm -rf $(OBJ_DIR) $(TARGET)
27
28 .PHONY: all clean deps
```

Listing 4: Makefile do projeto SensorLog

3 Casos de Uso

O sistema apresenta 8 opções no menu principal. A Figura 1 mostra a interface principal do SensorLog em execução no terminal.

```

xxtigas@nitro5 ~/Projects/faculdade/LingProg/sensorlog git:(main ?6 -5)
o (0) > ./sensorlog
Iniciando SensorLog...

  SensorLog v1.0

  1. Cadastrar sensor
  2. Importar leituras (CSV)
  3. Listar leituras
  4. Estatísticas de um sensor [Py]
  5. Detectar anomalias [Py]
  6. Gerar grafico de historico [Py]
  7. Remover leitura por ID
  8. Sair

Opcao: 
```

Figura 1: Menu principal do SensorLog em execução.

Para os testes a seguir, utilizou-se o arquivo `./data/exemplo.csv`, que contém medições de três sensores distintos (`s1` – temperatura, `s2` – umidade, `s3` – luminosidade) e inclui deliberadamente valores anômalos (como uma temperatura de 40.0°C e uma umidade de 95.0%). O conteúdo do arquivo é apresentado abaixo:

```

1 sensor_id;tipo;valor;unidade;timestamp
2 s1;temp;22.5;C;2026-07-01 08:00:00
3 s1;temp;23.1;C;2026-07-01 09:00:00
4 s1;temp;24.3;C;2026-07-01 10:00:00
5 s1;temp;23.8;C;2026-07-01 11:00:00
6 s1;temp;22.9;C;2026-07-01 12:00:00
7 s1;temp;40.0;C;2026-07-01 13:00:00
8 s1;temp;23.5;C;2026-07-01 14:00:00
9 s1;temp;24.1;C;2026-07-01 15:00:00
10 s2;hum;60.2;%;2026-07-01 08:00:00
11 s2;hum;61.5;%;2026-07-01 09:00:00
12 s2;hum;59.8;%;2026-07-01 10:00:00
13 s2;hum;62.0;%;2026-07-01 11:00:00
14 s2;hum;95.0;%;2026-07-01 12:00:00
15 s2;hum;60.9;%;2026-07-01 13:00:00
16 s3;lux;820.0;lx;2026-07-01 08:00:00
17 s3;lux;950.0;lx;2026-07-01 09:00:00
18 s3;lux;1100.5;lx;2026-07-01 10:00:00
19 s3;lux;870.0;lx;2026-07-01 11:00:00
20 s3;lux;100.0;lx;2026-07-01 12:00:00
21 s3;lux;910.0;lx;2026-07-01 13:00:00
```

Listing 5: Conteúdo do arquivo de teste exemplo.csv

3.1 Opção 1 – Cadastrar Sensor

Esta opção solicita ao usuário o ID do sensor e seu tipo (`temp`, `hum` ou `lux`), armazenando-o em um `std::map<string, Sensor>`. Rejeita duplicatas e tipos inválidos. A Figura 2 ilustra o processo de cadastro.

```
Opcao: 1

ID do sensor: 
```

(a) Programa solicita o ID e o tipo do sensor.

```
Opcao: 1

ID do sensor: s1
Tipo (temp / hum / lux): temp
[OK] Sensor 's1' cadastrado com sucesso.
```

(b) Confirmação do cadastro realizado com sucesso.

Figura 2: Opção 1 – Cadastro de um novo sensor.

3.2 Opção 2 – Importar Leituras (CSV)

Esta opção solicita o caminho de um arquivo `.csv` cujas linhas seguem o formato `sensor_id;tipo;valor;unidade;timestamp`. O C++ faz o parsing, valida cada linha individualmente e rejeita linhas com formato inválido ou sensores não cadastrados. A Figura 3 demonstra a importação.

```
Opcao: 2

Caminho do arquivo CSV: 
```

(a) Programa solicita o caminho do arquivo CSV.

```
Opcao: 2

Caminho do arquivo CSV: ./data/exemplo.csv
Importacao concluida: 20 leitura(s) importada(s), 0 linha(s) rejeitada(s).
```

(b) Resultado da importação com contagem de leituras.

Figura 3: Opção 2 – Importação de leituras via arquivo CSV.

3.3 Opção 3 – Listar Leituras

Esta opção exibe todas as leituras armazenadas em uma tabela formatada contendo ID, sensor, tipo, valor, unidade e timestamp. Não há interação adicional com o usuário. A Figura 4 mostra a listagem.

3.4 Opção 4 – Estatísticas de um Sensor [Python]

Esta opção solicita o ID de um sensor cadastrado, coleta seus valores, envia ao Python via pipe e recebe de volta os descritores estatísticos: média, mediana, desvio padrão, mínimo, máximo e coeficiente de variação. A Figura 5 ilustra o processo.

Opcao: 3

ID	Sensor	Tipo	Valor	Unidade	Timestamp
1	s1	temp	22.50	C	2026-07-01 08:00:00
2	s1	temp	23.10	C	2026-07-01 09:00:00
3	s1	temp	24.30	C	2026-07-01 10:00:00
4	s1	temp	23.80	C	2026-07-01 11:00:00
5	s1	temp	22.90	C	2026-07-01 12:00:00
6	s1	temp	40.00	C	2026-07-01 13:00:00
7	s1	temp	23.50	C	2026-07-01 14:00:00
8	s1	temp	24.10	C	2026-07-01 15:00:00
9	s2	hum	60.20	%	2026-07-01 08:00:00
10	s2	hum	61.50	%	2026-07-01 09:00:00
11	s2	hum	59.80	%	2026-07-01 10:00:00
12	s2	hum	62.00	%	2026-07-01 11:00:00
13	s2	hum	95.00	%	2026-07-01 12:00:00
14	s2	hum	60.90	%	2026-07-01 13:00:00
15	s3	lux	820.00	lx	2026-07-01 08:00:00
16	s3	lux	950.00	lx	2026-07-01 09:00:00
17	s3	lux	1100.50	lx	2026-07-01 10:00:00
18	s3	lux	870.00	lx	2026-07-01 11:00:00
19	s3	lux	100.00	lx	2026-07-01 12:00:00
20	s3	lux	910.00	lx	2026-07-01 13:00:00

Figura 4: Opção 3 – Listagem de todas as leituras armazenadas.

```
Opcao: 4

Sensores disponiveis:
- s1 [temp]
- s2 [hum]
- s3 [lux]
Informe o ID do sensor: █
```

(a) Programa lista os sensores e solicita o ID.

```
Opcao: 4

Sensores disponiveis:
- s1 [temp]
- s2 [hum]
- s3 [lux]
Informe o ID do sensor: s1

=== Estatisticas: s1 [temp] ===
Media           : 25.5250
Mediana         : 23.6500
Desvio Padrao   : 5.6810
Minimo          : 22.5000
Maximo          : 40.0000
Coef. de Variacao : 22.2567 %
```

(b) Estatísticas retornadas pelo módulo Python.

Figura 5: Opção 4 – Cálculo de estatísticas via módulo Python.

3.5 Opção 5 – Detectar Anomalias [Python]

Esta opção solicita o ID do sensor e o limiar k de z-score (padrão 2.0). Os valores são enviados ao Python, que calcula os z-scores e retorna os índices das leituras que desviam mais de $k\sigma$ da média. A Figura 6 mostra a detecção de anomalias.

```
Opcao: 5

Sensores disponiveis:
- s1 [temp]
- s2 [hum]
- s3 [lux]
Informe o ID do sensor: █
```

(a) Programa solicita o sensor e o limiar k .

```
Opcao: 5

Sensores disponiveis:
- s1 [temp]
- s2 [hum]
- s3 [lux]
Informe o ID do sensor: s1
Limiar k (z-score, padrao=2.0): 1

=== Anomalias: s1 (k=1.0000) ===
Idx  Valor  Z-score  Timestamp
-----
5    40.00   2.548    2026-07-01 13:00:00
13   40.00   2.548    2026-07-01 13:00:00
```

(b) Anomalias detectadas com índices e z-scores.

Figura 6: Opção 5 – Detecção de anomalias via z-score (módulo Python).

3.6 Opção 6 – Gerar Gráfico de Histórico [Python]

Esta opção solicita o ID do sensor, envia os valores e timestamps ao Python, que gera um gráfico de linha em PNG utilizando `matplotlib` e retorna o caminho do arquivo gerado. A Figura 7 ilustra o processo.

```
Opcao: 6

Sensores disponiveis:
- s1 [temp]
- s2 [hum]
- s3 [lux]
Informe o ID do sensor: █
```

(a) Programa solicita o ID do sensor.

```
Opcao: 6

Sensores disponiveis:
- s1 [temp]
- s2 [hum]
- s3 [lux]
Informe o ID do sensor: s1
[OK] Grafico gerado: /tmp/sensorlog_4xrh3bqc.png
```

(b) Caminho do gráfico PNG gerado pelo Python.

Figura 7: Opção 6 – Geração de gráfico de histórico via Python.

A Figura 8 apresenta um exemplo de gráfico gerado pelo sistema para o sensor `s1`.

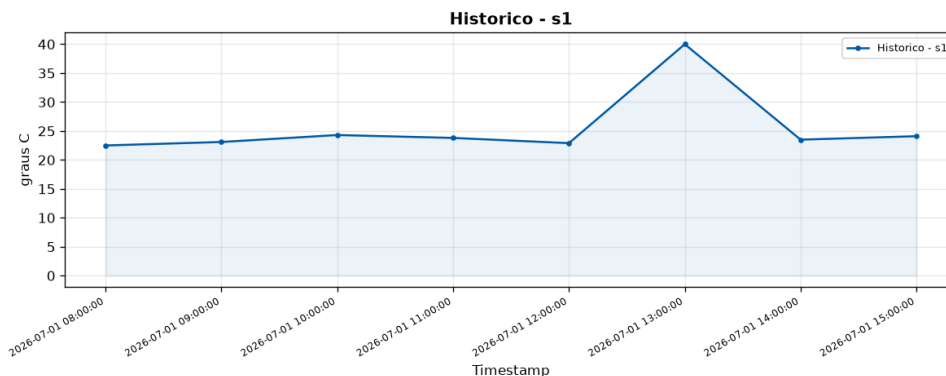


Figura 8: Gráfico de histórico gerado pelo módulo Python via `matplotlib`.

3.7 Opção 7 – Remover Leitura por ID

Esta opção solicita o ID numérico de uma leitura e a remove do vetor interno. Exibe confirmação ou erro caso o ID não seja encontrado. A Figura 9 demonstra a remoção.

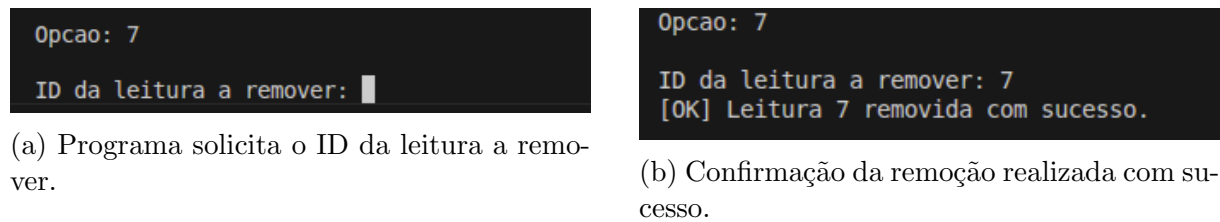


Figura 9: Opção 7 – Remoção de uma leitura por ID.

3.8 Opção 8 – Sair

Esta opção encerra o programa com uma mensagem de despedida. A Figura 10 mostra a saída do sistema.

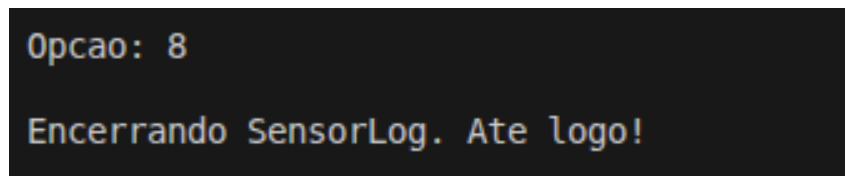


Figura 10: Opção 8 – Encerramento do programa.

4 Conclusão

A implementação da Etapa 2 do trabalho foi concluída com sucesso. O sistema SensorLog atendeu a todos os requisitos exigidos: utiliza orientação a objetos de forma extensa em C++, não utiliza a função `system()`, não utiliza arquivos para troca de parâmetros, e é robusto contra entradas inválidas. A interface via `popen()` e JSON mostrou-se uma solução elegante e eficiente para integrar C++ e Python, permitindo isolar as regras de negócio (gerenciamento de sensores e leituras) das análises matemáticas complexas (estatísticas, detecção de anomalias e visualização). O uso do Python provou ser altamente vantajoso para as tarefas de cálculo estatístico e geração de gráficos, validando a premissa de que diferentes linguagens devem ser combinadas para extrair o melhor de cada ecossistema.

Referências

- [1] DTI Digital, “Matplotlib: como gerar gráficos usando Python”. Disponível em: <https://www.dtidigital.com.br/blog/matplotlib-como-gerar-graficos-usando-python>. Acessado em julho de 2026.
- [2] Python Academy, “Como criar gráficos utilizando Matplotlib no Python”. Disponível em: <https://pythonacademy.com.br/blog/como-criar-graficos-utilizando-matplotlib-no-python>. Acessado em julho de 2026.

- [3] PM3, “Python para Análise de Dados”. Disponível em: <https://pm3.com.br/blog/python-para-analise-de-dados/>. Acessado em julho de 2026.